A

International Institute of Information Technology, Hyderabad

(Deemed to be University)

CS3.301 Operating Systems and Networks – Monsoon 2023

Quiz 2

Max. Time: 45 mins **Max. Marks: 20**

Roll No: _____ **Programme: _____**

Student's Signature: _____ **Invigilator's Signature:_____**

Special Instructions to the students

1. Answers written with pencils won't be considered for evaluation
2. Please **read the descriptions** of the questions (scenarios) carefully.
3. There are a total of six questions with four MCQs and carries 20 marks. For MCQs you can select one/all that applies as answers for a given MCQ. Please refrain from writing long explanations for MCQ questions. **Keep the explanations short and to the point (4-5 lines).**
4. **Feel free to use extra page** for any calculations/rough work but **they won't be considered for evaluations**.

Marks Table (To be filled by the Evaluator)

Question No / Marks	Initial	Final	Name of the Evaluator
1			
2			
3			
4			
5			
6			

General Instructions to the students

1. Place your Permanent / Temporary Student ID card on the desk during the examination for verification by the Invigilator.
2. Reading material such as books (unless open book exam) are not allowed inside the examination hall.
3. Borrowing writing material or calculators from other students in the examination hall is prohibited.
4. If any student is found indulging in malpractice or copying in the examination hall, the student will be given 'F' grade for the course and may be debarred from writing other examinations.

Best of Luck

Welcome to the Second Round of the Operating Systems and Networks Design Competition. Following the previous round, the OSNT team of system designers implemented process and memory virtualization capabilities into their operating system. They are now intending to add concurrency support as well as support for a few application layer protocols. They are seeking for new team members in this area. They wanted to put you to the test to determine if you could join their team. You have 45 minutes to respond to some questions they have for you. Each question is worth a certain number of points, and the total number of points you obtain determines your place on the team.

1. The team believes that they have built its operating system with support for concurrency. To that purpose, the team has enabled the use of locks and condition variables via the POSIX library (*pthread_lock_t* for declaring locks, *pthread_cond_t* for declaring condition variables). The team wants you to put their solution to the test. As part of this, the team has asked you to solve a synchronization problem where there are two different types of threads: *In* and *Out*. Threads of type *In* keeps writing integer values to a buffer of size 10, while other threads of type *Out* keeps printing the values written to the buffer. An *Out* threads can fail if it attempts to print when the buffer is empty, and the *In* threads can fail if they attempt to write values to a full buffer. The team expects you to write a pseudocode in C and reason on it. **(4 points)**

2. The team deployed its UNIX-based operating system on one of the server machines, which also serves as a DNS server (particularly TLD server) for resolving various sorts of domains such as .com, .net, and .in. They however, want to confirm with you how the DNS records for such domains are stored so that they can be sent as a response to a query from a local DNS. Which of the following do you suggest and why? **(3 points)**
   A. (sampleweb.com, ns.sampleweb.com, NS, 86400)
   B. (sampleweb.com, 122.148.21.21, A, 3600)
   C. (sampleweb.com, 122.148.21.21, AAAA, 3600)
   D. (sampleweb.com, mail.sampleweb.com, MX, 86400)

3. The team is considering providing support for using semaphores in the OS. But, they would like to know from you how such a semaphore can be used to solve synchronization problems. To this end, they want you to write and explain briefly a simple pseudo code (in C) for a parent program that creates a child thread and waits for the child thread to complete before the parent program exits using the concept of semaphores. The child thread simply prints “I am child” on execution and parent prints “completed” upon execution. Your code needs to ensure that “I am child” is always printed prior to printing “completed” when running the program. **(4 points)**

4. The team has implemented the notion of semaphores in their UNIX based OS. However, they further wanted to test by putting it to use to solve some classical problem. They have come up with the following snippet to solve the producer consumer problem. The idea is to use a buffer of size 10 integers. They want your opinion on if the written snippet contains some synchronization issues or if it is good to be deployed. What would be your opinion and why? **(3 points)**

Producer

```
void *producer(void *arg)
{
    ... //for loop
    sem_wait (&empty);
    sem_wait (&mutex);
    put(i);
    sem_post (&mutex);
    sem_post (&full);
}
```

Consumer

```
void *consumer(void *arg)
{
    .. //for loop
    sem_wait (&full);
    sem_wait (&mutex);
    int tmp = get();
    sem_post (&mutex);
    sem_post (&empty);
}
```

A. This code can result in a deadlock
B. This program does not have any synchronization issues
C. This program will work only if there is one producer and one consumer
D. None of the above

5. The team is looking to build application layer support in their OS network stack. The team is considering to use HTTP/1.1 and thereby built an HTTP server, but since HTTP is stateless they are not sure how to store some necessary client information. What do you recommend? Please mark your option followed by explanation on how it can help address the concern **(3 points)**
    A. The team can make use of cookies
    B. Web caches can be leveraged
    C. The team should use HTTP/1.0 instead
    D. Make use of CDNs

6. The team has developed their OS ensuring that some hardware primitives are available for locking critical section during concurrent access. In order to test it, they are planning to write a program that creates ten different threads which should concurrently increment a shared integer variable, *balance* by 1 (*balance = balance + 1*). But they are not very clear which of the following locks they should use. Which one do you recommend and explain why? **(3 points)**
    A. Test-and-Set
    B. Compare-and-Swap
    C. Park and unpark
    D. Fetch and Add

Now that you have completed the task, it's time to wait to know your final points!!!

**********Extra Space*********